

Quantization, Structured Outputs and Tool Calling, from the Physics

Inference Fundamentals, Part 4: the roofline, what quantization actually saves, how a schema becomes a token mask, and why an agent's first token keeps getting slower

Parts 1B through 3 built the model of an LLM request from the hardware up: two phases, two limits, a knee, a cliff, and a set of metrics that fall out of them. This part uses that model on four subjects that come up in nearly every serving conversation and that the earlier parts only named: the roofline chart that holds the whole model on one axis, quantization and what each format does and does not save, structured outputs, and tool calling. It ends with the arithmetic of tail latency in agent workflows and a one-page diagnostic tree. Nothing here needs a new principle. Everything is the same division, applied.

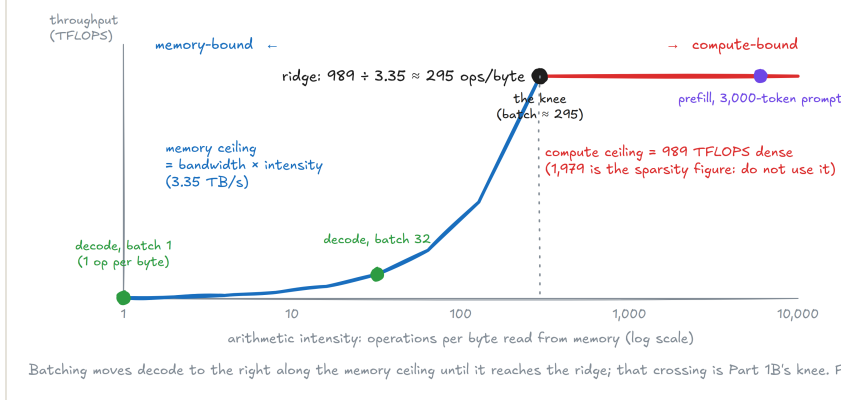
Where does the time go? For more than a decade that was my question, asked of large data platforms on behalf of enterprises: was a query reading, computing, or waiting for a slot; was the compute sized for the concurrency the system actually saw or the concurrency someone guessed; what deserved a cache; and what did all of it cost per query, in terms finance could act on. More recently the question has moved to language models: agentic systems, and the open-source and frontier models beneath them. I came to inference expecting a new set of rules. What I found was the same discipline in different units: two hardware limits, a handful of formulas connecting them to what a user feels, and the familiar problem of which lever a given symptom is asking for. This series is for readers with that background who want to understand inference well enough to run it.

The subjects in this part are the ones that separate a serving conversation from a serving decision. A customer rarely asks "is decode memory-bound." They ask whether to quantize, whether the JSON will be valid, why their agent gets slower with every turn, and why one in ten user actions is slow when every model call is fast. Each of those has an answer that can be derived rather than recalled, and deriving it is the point.

The roofline: the whole series on one chart

Part 1B established that decode reads every weight once per step and does two operations with each, and that prefill does the same read but amortizes it across every prompt token. The roofline model, which predates language models by a decade and a half, turns that observation into a chart. On the horizontal axis is arithmetic intensity: the number of floating-point operations a piece of work performs for every byte it reads from memory. On the vertical axis is the throughput the hardware can deliver for work of that intensity. The chart has two ceilings. One is flat, at the GPU's peak arithmetic rate. The other slopes upward from the origin, because work of low intensity is limited by how fast bytes arrive, and its ceiling is bandwidth times intensity. The two meet at a point called the ridge.

The roofline: one axis for every workload (H100, dense FP16)



The ridge is a single division, and for an H100 it is worth memorizing: 989 TFLOPS of dense FP16 arithmetic divided by 3.35 TB/s of bandwidth gives about 295 operations per byte. Work that does fewer than 295 operations per byte read is memory-bound; work that does more is compute-bound. In FP8 the arithmetic peak doubles to 1,979 TFLOPS and the ridge moves to about 591.

Now place the two phases. A decode step at batch 1 with FP16 weights reads two bytes per weight and does two operations with it: one operation per byte, three hundred times to the left of the ridge. Every request added to the batch adds two operations per weight without adding a byte, so intensity grows with the batch, and the step reaches the ridge at a batch of about 295. That is the knee from Part 1B, seen from the other side. Prefill on a 3,000-token prompt has an intensity of thousands of operations per byte and sits far to the right, under the flat ceiling, which is why it is compute-bound from the first token. The chart is the same model as Part 1B's "two budgets, take the larger"; it just puts every workload on one axis so you can see where each one lands.

Two practical rules fall out of the chart. The first is a trap: NVIDIA's datasheets lead with sparsity-enabled figures, 1,979 TFLOPS for FP16 and 3,958 for FP8 on an H100, and those are twice the dense numbers. LLM inference does not use structured sparsity, so the dense figure is the one to divide by. Quoting 1,979 as the FP16 peak halves your ridge point and every estimate that depends on it. The second is a proof you can do in your head. An H200 has exactly the same arithmetic as an H100, 989 TFLOPS, but 4.8 TB/s of bandwidth instead of 3.35, and it decodes the same model about 1.4 times faster. That is 4.8 divided by 3.35. Compute did not change at all, which is exactly what a workload three hundred times to the left of the ridge predicts, and the B200's 8 TB/s continues the same line.

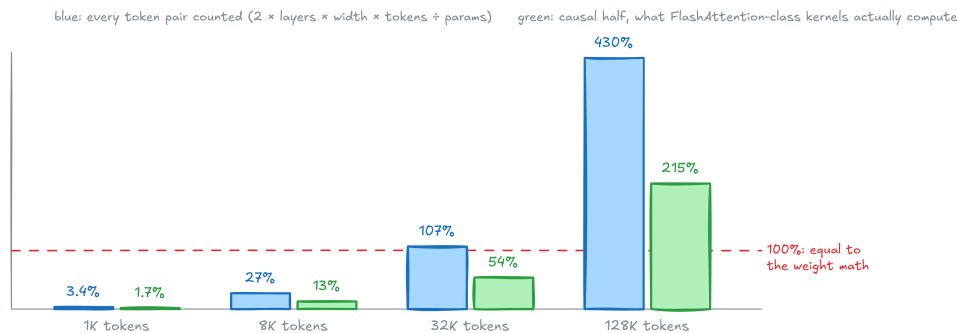
One more term belongs here because it turns the flat ceiling into a number you can use. Model FLOPs utilization, MFU, is the fraction of the dense peak that a real forward pass achieves; kernel launch gaps, attention's own irregular arithmetic, and memory traffic that is not weights all pull it down. Prefill on well-tuned engines reaches somewhere between 35 and 55 percent of peak. For an 8B model on a 1,000-token prompt, which is 16 trillion operations, that is 29 to 46 milliseconds of pure arithmetic. When a benchmark reports a prefill time, MFU is the number that tells you whether the engine is doing well or leaving half the GPU idle.

Attention's second term

The two-operations-per-weight rule counts the arithmetic that involves weights. Part 1B noted that attention, which compares each token against every earlier one, is left out of that count and is small until contexts get long. Here is how long.

Attention's arithmetic per layer per token is proportional to the context length times the model's hidden width: a score against every earlier token, and a weighted sum over every earlier token's value. For the whole prompt that is proportional to the square of the prompt length, which is where "attention is quadratic" comes from. For Llama 3 8B, with 32 layers and a hidden width of 4,096, the ratio of attention arithmetic to weight arithmetic is easy to compute, and the picture below shows it at four prompt lengths.

Attention arithmetic as a share of the weight arithmetic (Llama 3 8B: 32 layers, width 4,096)



Below 8K tokens the weight count is a fine estimate of prefill cost. Above 32K it is badly wrong: the attention kernel, not the matrix multiply, is where long-context effort goes.

Counting every pair of tokens, attention is about 3 percent of the weight math at 1,000 tokens, 27 percent at 8,000, roughly equal at 32,000, and four times the weight math at 128,000. Modern attention kernels skip the half of the pairs that the causal mask forbids, so the work actually done is about half of those figures, and the crossover on this model lands nearer 64,000 tokens. Either way the shape is what matters: below 8,000 tokens the weight count is a fine estimate of prefill cost, and above 32,000 it is badly wrong. That is why a 128K-context request costs far more than 128,000 times a short request's per-token price, why prefill time for very long prompts grows faster than linearly, and why the attention kernel, not the matrix multiply, is where long-context engineering effort goes.

Chunked prefill: the dial between the two phases

Part 2 described chunked prefill as one of three answers to the "prefill blocks decode" problem. Now that prefill has a number, the mechanism can be made precise.

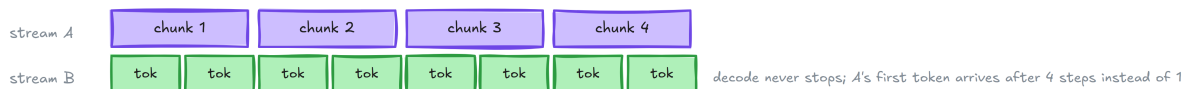
The engine scheduler has a budget of tokens per step, in vLLM called `max_num_batched_tokens`. At every step it first admits every running request's next decode token, one each, and then spends whatever budget remains on prefill, slicing a long prompt into chunks that fit. A 32,000-token prompt on a budget of 8,192 takes four steps to prefill instead of one long one, and the decode steps of every other request continue in between. Without chunking, that single prompt would occupy the GPU for a second or more of continuous arithmetic on a large model while every other user's stream froze.

Chunked prefill: the per-step token budget (`max_num_batched_tokens`)

Without chunking

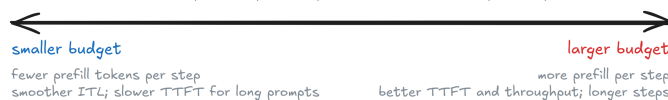


With chunking (budget 8,192 per step)



The dial

vLLM defaults (server / offline): H100, H200: 8,192 / 16,384 · A100 and smaller: 2,048 / 8,192 · 160 GB+ GPUs: 16,384



512, from early documentation, sits at the ridge point and starves throughput. Related caps: max partial prefills, long-prefill token threshold.

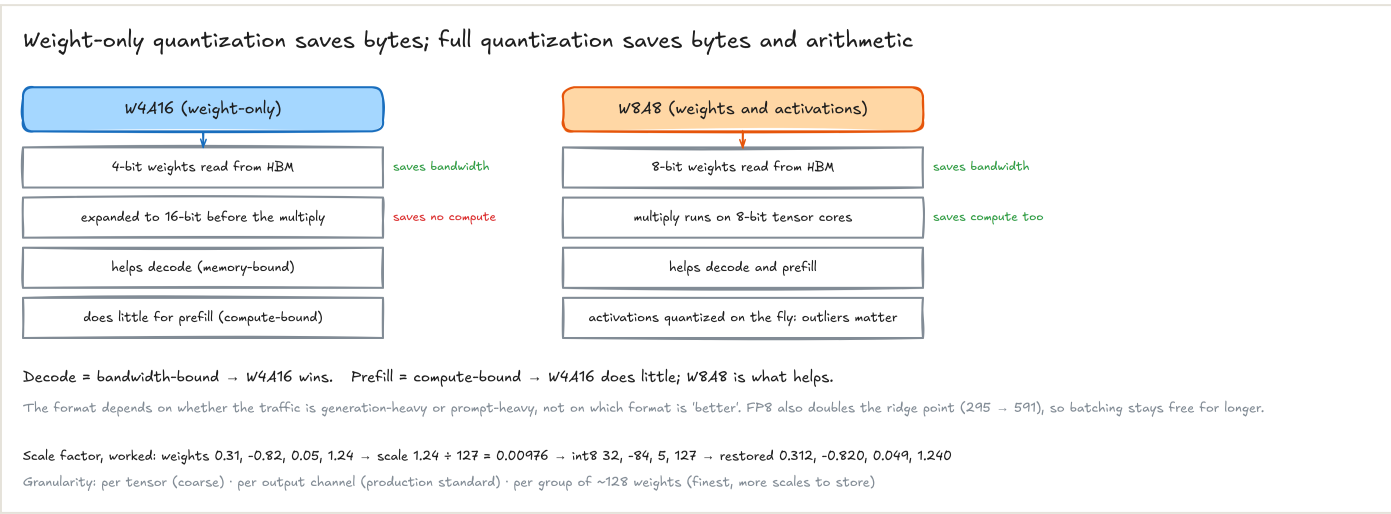
The budget is a dial with one number on each end. A smaller budget means fewer prefill tokens interrupt each decode step, so inter-token latency stays smooth, at the cost of long prompts taking more steps to finish and therefore a longer time to first token for those requests. A larger budget packs more prefill into each step, which brings the first token sooner and raises throughput, at the cost of longer steps for everyone decoding. There is no right value; there is the value that meets your SLO for your traffic. The defaults are hardware-dependent, which is worth knowing because stale numbers circulate. In current vLLM, chunked prefill is on by default and the budget defaults to 8,192 tokens for the API server and 16,384 for the offline class on H100 and H200; on A100 and smaller GPUs the defaults are 2,048 and 8,192; on GPUs with 160 GB or more it is 16,384 for both. The figure of 512 that appears in older documentation is from an early version and is far too small for modern GPUs: it sits right at the ridge point, so a budget that low starves throughput while giving little in return. Two related settings cap how many partial prefills can run at once and how long a prompt must be before it is chunked, so that a swarm of medium prompts cannot starve a long one.

Quantization: what each format saves

Part 1B showed that halving the bytes per weight halves the decode floor, and Part 2 listed quantization as the lever that attacks bytes per step. What that summary hides is that "quantization" is several different decisions, and the right one depends on which phase dominates the workload.

Start with the mechanism. A weight stored in 16-bit floating point takes two bytes. Quantization stores it in fewer bits plus a scale factor that maps the small integers back to the original range. Take four weights, 0.31, negative 0.82, 0.05 and 1.24. The largest magnitude is 1.24, and dividing it by 127, the largest 8-bit signed integer, gives a scale of about 0.00976. Dividing each weight by the scale and rounding gives 32, negative 84, 5 and 127, one byte each. Multiplying back gives 0.312, negative 0.820, 0.049 and 1.240: the original values with a small rounding error. The scale can be shared by a whole matrix, which is coarse and cheap, by each output channel, which is the production standard, or by every group of 128 or so weights, which is the finest and costs a little storage for the extra scales.

The notation tells you what was quantized, and it decides everything. W8A8 means weights in 8 bits and activations in 8 bits. W4A16 means weights in 4 bits and activations left at 16, which is called weight-only quantization. Activations are the intermediate values that flow between layers; they depend on the input, so they must be quantized on the fly, and they are the harder half.



Here is the consequence, and it is the whole strategy question. Weight-only quantization shrinks what is read from memory, but the arithmetic still happens in 16-bit, because the weights are expanded back before the multiply. It therefore speeds up the memory-bound phase, decode, and does almost nothing for the compute-bound phase, prefill. Quantizing activations as well lets the multiply run on the 8-bit tensor cores, which on an H100 are twice as fast, so W8A8 speeds up both phases. A generation-heavy workload, chat and agents producing long answers, gets most of its benefit from 4-bit weights. A prompt-heavy workload, summarization and RAG over long documents, gets little from weight-only quantization and needs the activations quantized too. The question is never which format is better; it is which phase the customer's traffic lives in.

Weights are easy to quantize because they are a fixed, known distribution: quantize once, offline, and inspect the result. Activations are hard because of outliers. A few channels carry values far larger than the rest, and since the scale is set by the maximum, one outlier stretches the range and crushes the ordinary values into a handful of levels. The named methods are each an answer to that problem. Round-to-nearest just rounds, and works for FP8 because a floating-point format handles wide ranges natively. GPTQ quantizes a matrix column by column and adjusts the remaining weights to compensate for the error already introduced; it needs a small calibration set. AWQ finds the small fraction of weight channels that matter most, judged by activation magnitude, and scales them to protect them. SmoothQuant migrates the difficulty from the activations into the weights, since the weights are the easy side.

On accuracy, there is one study large enough to lean on. An evaluation of more than 500,000 benchmark runs across the Llama 3.1 family at 8B, 70B and 405B found that FP8 weights and activations are effectively lossless at every scale, that well-tuned INT8 weights and activations lose about 1 to 3 percent, and that 4-bit weight-only quantization is more competitive than expected, rivaling the 8-bit formats. Its deployment guidance follows the physics above: 4-bit weight-only is the most cost-efficient for synchronous, low-concurrency serving, where decode dominates, and W8A8 wins for asynchronous continuous batching, where the batch pushes the step toward the compute side. What that study does not license is the word "free." The sentence to carry is that FP8 is lossless on published benchmarks across the Llama 3.1 family, which is a strong prior, and that you still validate on the customer's own evaluation set because their task distribution is not the benchmark distribution.

One subtlety of FP8 is worth raising unprompted. Doubling the arithmetic peak doubles the ridge point, from about 295 to about 591 operations per byte, so FP8 pushes more of the batch range into the memory-bound regime even as it halves the bytes moved. A workload that was just past the knee in FP16 is comfortably to the left of it in FP8, which means batching stays free for longer. That is a second reason FP8 is close to pure gain on Hopper and later, beyond the halved floor.

The KV cache is a separate lever, with its own switch. Part 1B derived the cache at 320 KB per token for Llama 3 70B and showed that bytes per value is the one factor an operator controls. Storing the cache in 8 bits halves every figure, which doubles the batch or the context that fits on the same GPU, and that feeds directly back into the batching win of Part 2. FP8 and INT8 caches are supported in the major engines and carry little task-level cost for most workloads; 4-bit caches are more sensitive, and reasoning and code are the tasks most likely to drift. Weight precision and cache precision are independent choices, and an engine can run FP8 weights with a 16-bit cache or the reverse depending on which memory it is short of.

Structured outputs: how a schema becomes a mask

When an application needs JSON that matches a schema, or output that follows a grammar, the reliable way to get it is not to ask nicely and retry. It is to constrain decoding: at every step, before sampling, mask out every token that would make the output invalid given what has been produced so far. The grammar is compiled from the schema into an automaton, the automaton tracks the state of the output, and each step produces a mask over the vocabulary.

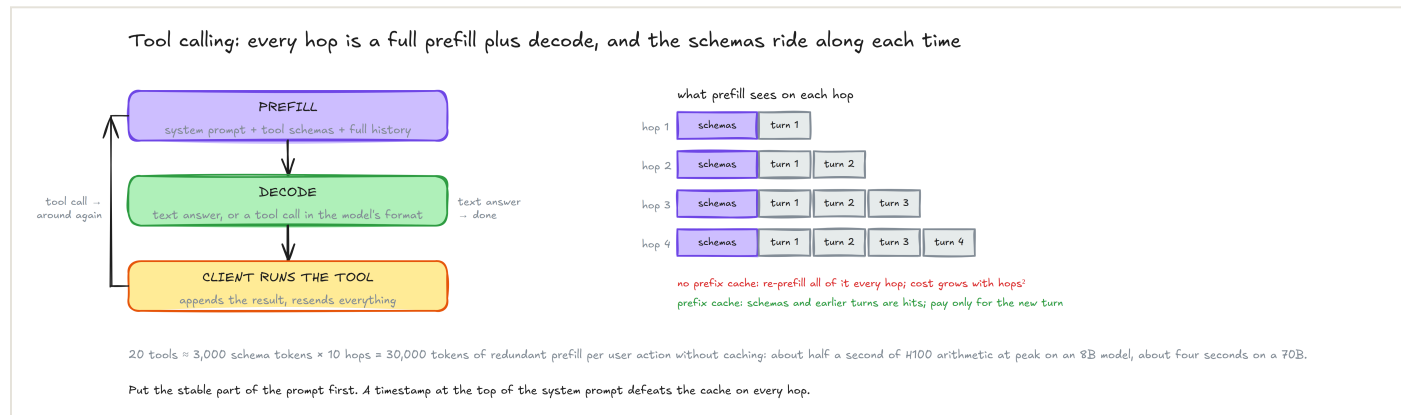
The reason this was expensive and no longer is comes down to a count. Checking every token in a 128,000-entry vocabulary against the grammar at every step is a lot of work. The observation behind the current engines is that for a given grammar state, the validity of almost every token does not depend on context. For Llama 3.1's vocabulary under a JSON grammar, only about 1,100 tokens out of 128,000 need to be checked at runtime; the rest can be classified once, ahead of time, as always accepted or always rejected in that state. With that precomputation the per-token mask costs tens of microseconds for JSON, which is nothing next to a decode step, and the leading implementation reports more than a hundredfold speedup over earlier approaches. It is the default structured-output backend in the major open-source engines. Treat the exact speedup figures as the project's own claims; the mechanism is the part you can assert.

The enterprise framing is a cost argument, not a quality one. Unconstrained models already produce schema-valid JSON most of the time on well-specified tasks; constrained decoding raises that toward certainty. What it eliminates is the validate-and-retry loop, and each retry is a complete extra inference call: a full prefill and a full decode, billed and queued like any other. A workload with a 5 percent retry rate is paying 5 percent more and adding a full request's latency to one call in twenty, and a tail that includes retries is the tail the user notices.

Tool calling is structured outputs plus a loop

Function calling is where the earlier parts all arrive at once, which is why it is the workload that stresses a serving stack hardest. Here is what actually happens on the wire.

The client sends the user's message plus a list of tool definitions: a name, a description, and a JSON schema for the arguments. The chat template renders those definitions into the prompt as ordinary tokens. There is no tools API inside the model; there is a prompt with the schemas written into it. The model then decides whether to answer in text or to emit a tool call, and if it calls, it generates the call in its own family's format. The server's parser recognizes that format and converts it into the standard tool-call JSON that applications expect. The client runs the tool, appends the result as a tool message, and sends the whole conversation back. That is one hop, and the loop runs until the model answers in text.



Every hop is a full prefill plus a full decode, and the schemas are tokens in the prompt on every one of them. A realistic agent with twenty tools carries roughly 3,000 tokens of schema on each hop, on top of a conversation that grows with every tool result. Over ten hops that is 30,000 tokens of schema alone, prefilled ten times, plus the prior turns re-sent on each hop, so the prefill bill grows with the square of the hop count. On an 8B model, 30,000 tokens is about half a second of H100 arithmetic at peak per user action; on a 70B model it is about four seconds. Without prefix caching, time to first token climbs turn over turn, which users report as "the agent gets slower the longer I use it." With prefix caching, from Part 2, the schema block and the prior turns are cache hits and each hop pays prefill only on the new tool result. This is the concrete reason prefix caching is the single highest-leverage setting in agentic serving, and it is also why the stable part of the prompt, the schemas and the system prompt, must come first: a timestamp at the top of the system prompt defeats the cache for every hop.

There is no single format for emitting a call. Each model family was trained on its own: XML-style tags wrapping JSON for the Hermes and Qwen families, a control token followed by a JSON array for Mistral, raw JSON for Llama 3.1 and 3.3, and a pythonic function-invocation syntax for Llama 3.2 and Llama 4. The serving engine needs a parser that matches the model; in vLLM that is an auto-tool-choice flag and a parser name such as `hermes`, `mistral`, `llama3_json` or `llama4_pythonic`, and all of them are normalized to the same client-facing JSON so application code never sees the difference. Mismatching the parser and the model is the most common "tool calling is broken" ticket: the model's calls come back as plain text.

The `tool_choice` field is where structured outputs become production behavior. `auto` lets the model decide, `none` forbids calls, `required` demands at least one, and naming a specific function demands that one. The last two are implemented with constrained decoding against the tool's schema, so the argument JSON is guaranteed to parse and to conform. Whether a model can emit several calls in one turn matters too: parallel calls cut the hop count, which cuts latency and redundant prefill with it, and support is model-specific. Llama 3 does not do parallel calls; Llama 4 does. Ask about it when sizing an agent workload, because it changes the hop count that feeds the tail math below.

The failure modes are worth listing because each has a different fix. A hallucinated tool name, a function that does not exist, is not fixed by constraining the arguments; only constraining the name field does, and a named or required `tool_choice` is the lever. Wrong argument types, an array serialized as a string or a number as a string, are a documented failure even on well-supported models, and schema-constrained arguments fix them. Malformed JSON, truncated at the token limit or with unbalanced braces, has the same fix plus a sane

maximum output length. Calls arriving as text is the parser flag. And a fine-tuned model that has lost its tool format is a chat-template problem, visible in the tokenizer configuration. One more thing is real engineering rather than configuration: streaming a tool call. Streaming text is trivial, but a client needs a complete, parseable object, and tokens arrive one at a time, so the server has to parse JSON incrementally and emit partial deltas until the braces balance. Parsers differ in how well they do this, and it is a fair question to ask of any vendor.

Function calling is the place where a model's output becomes a side effect in someone's system, so the reliability bar is higher than for chat. The customer's real question is never whether tools are supported. It is what the schema-adherence rate is under load and what happens on the failures.

Tail latency in an agent, with the arithmetic

Part 3 explained why the 99th percentile explodes while the median barely moves: queueing. Agents add a second mechanism that has nothing to do with queueing, and it is pure arithmetic.

Dean and Barroso's "The Tail at Scale" made the observation for web search: when one user request fans out to 100 backends and each backend is slow one time in a hundred, the user request is slow whenever any backend is, which is $1 - 0.99^{100}$ to the hundredth power, about 63 percent of the time. Agent workflows are the serial version of the same thing. A user action that makes ten model calls in sequence hits a 1-in-100 tail on at least one of them with probability $1 - 0.99^{10}$ to the tenth, about 9.6 percent. One user action in ten hits the tail, not one in a hundred, even though every individual call has a perfectly good p99. At twenty hops it is 18 percent. That is the sentence for a CTO: the p99 of the model is not the p99 of the product, and the exponent is the hop count.

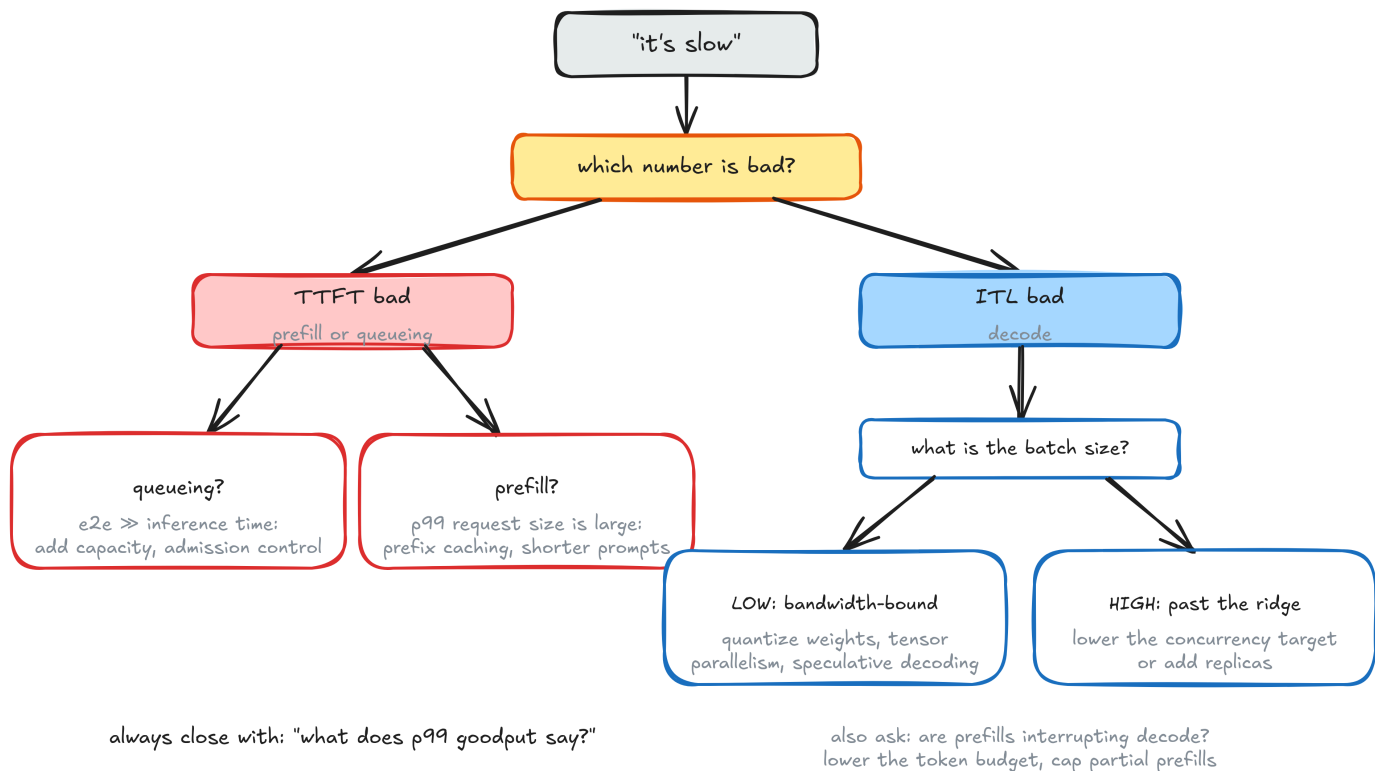
The corollary is that in agentic serving the per-call tail has to be far tighter than the product's tail, or the hop count has to come down, through parallel tool calls, fewer tools, or a smaller model for the routing steps. And tail latency compounds through the client: slow calls trigger timeouts, timeouts trigger retries, and retries add load, which pushes the tail further out. Close on cost, not feel.

Two calibrations belong with this. First, know which metric a number is for. A target of "50 milliseconds at p99" is not a plausible end-to-end time to first token, which lives in the hundreds of milliseconds; it is an inter-token budget, roughly twenty tokens per second of perceived stream speed. Inter-token targets in the literature run from a tight 30 milliseconds to a generous 100. Recognizing that a number belongs to ITL rather than TTFT is itself a signal. Second, measure with the right load generator. A closed-loop generator issues the next request only after the previous one completes, so an overloaded system automatically receives less load and its tail looks artificially healthy. An open-loop generator issues requests on the schedule the real traffic would, and lets the queue form. Goodput, from Part 3, measured under open-loop load against p99 TTFT and p99 ITL targets, is the number that says whether a configuration change worked.

The diagnostic tree

Everything in the series collapses to a tree you should be able to narrate without notes.

The diagnostic tree



"It's slow" is not a symptom; the first question is which number is bad. If time to first token is bad, the cause is prefill or queueing. Compare end-to-end time against inference time and check request size at the 99th percentile: if the gap is queueing, add capacity or admission control; if it is prefill, prefix caching, shorter prompts, or a larger prefill budget. If inter-token latency is bad, the cause is decode, and the next question is the batch size. At low batch the step is bandwidth-bound and the levers are quantized weights, tensor parallelism, and speculative decoding. At high batch the step is past the ridge and the levers are a lower concurrency target or more replicas. In either branch, ask whether prefills are interrupting decode, and if so lower the token budget or cap partial prefills. Always close with the same question: what does p99 goodput say?

The capacity calculation that goes with the tree takes two minutes out loud. An H100 has 80 GB. Llama 3.1 8B in FP8 is 8 GB of weights; allow 8 GB for the engine; 64 GB remains for the cache. At 128 KiB per token, the 131,072 bytes that Part 1B rounded to 128 KB, and a useful anchor is that 8,192 of them make exactly 1 GiB, that is about 500,000 tokens in flight. At 8,000 tokens per user, about 60 concurrent users per GPU. State the assumptions first, average context, average output, target inter-token latency, and then name the one the answer is most sensitive to, which here is context length. That is consulting, not trivia.

Check yourself

1. Should a customer with long summarization prompts use 4-bit weight-only quantization? It will not help much. Their workload is prefill-dominated and prefill is compute-bound; weight-only quantization reduces bytes read but not arithmetic done, because the multiply still runs in 16-bit. They need activations quantized too, W8A8, which uses the 8-bit tensor cores. A generation-heavy workload would be the opposite case.

2. Why does an agent's time to first token climb turn over turn, and what fixes it? Every hop re-sends the tool schemas and the whole prior conversation, and without prefix caching every one of those tokens is prefilled again, so prefill grows with the hop count and the total

grows with its square. Prefix caching makes the schemas and the earlier turns cache hits, so each hop pays prefill only for the new tool result. Put the stable part of the prompt first.

3. Every model call has a p99 of 800 ms, but users complain that one action in ten is slow. What is happening? The action makes about ten serial calls, and 1 minus 0.99 to the tenth is 9.6 percent. The product's tail is the model's tail raised to the hop count. Tighten the per-call tail, cut the hop count with parallel calls, or both.

4. What is the ridge point of an H100 and why does it matter? About 295 operations per byte in FP16, from 989 TFLOPS divided by 3.35 TB/s, and about 591 in FP8. Work below it is memory-bound and above it compute-bound. Decode at batch 1 sits at one operation per byte, which is why it is bandwidth-bound; batching raises intensity until it reaches the ridge, which is the knee. Derive it rather than quote it, and use the dense figure, not the sparsity one.

This closes the series. Part 1A explained what inference is and why it became everyone's problem; Part 1B derived the physics; Part 2 covered the engine; Part 3 the metrics and the operator's view; and this part the decisions that sit on top of them. If you can draw the roofline, place a workload on it, and say which lever its position is asking for, you have the model. The rest is measurement.

Radx Radhakrishnan works on LLM inference and agentic systems, after a decade of performance work on enterprise data platforms.

Sources

- Williams, Waterman and Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," Communications of the ACM, 2009. <https://dl.acm.org/doi/10.1145/1498765.1498785>
- NVIDIA, H100 Tensor Core GPU product page and datasheet (989 TFLOPS FP16 dense, 1,979 FP8 dense; sparsity figures are double; 3.35 TB/s). <https://www.nvidia.com/en-us/data-center/h100/>
- NVIDIA, Blackwell B200 specifications (8 TB/s HBM3e). <https://www.nvidia.com/en-us/data-center/dgx-b200/>
- Kaplan et al., "Scaling Laws for Neural Language Models," 2020, Section 2.1 (2N inference, 6N training operations per token). <https://arxiv.org/abs/2001.08361>
- Meta, Llama 3 model card and configuration (Llama 3 8B: 32 layers, hidden size 4,096, 8 KV heads). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022 (causal-mask skipping in attention kernels). <https://arxiv.org/abs/2205.14135>
- Agrawal et al., "SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills," 2023. <https://arxiv.org/abs/2308.16369>
- vLLM source, `vllm/engine/arg_utils.py` (hardware-dependent `max_num_batched_tokens` defaults: 8,192 server and 16,384 offline on H100/H200; 2,048 and 8,192 on A100 and smaller; 16,384 on GPUs with 160 GB or more; chunked prefill and prefix caching on by default), `vllm/config/cache.py` (`gpu_memory_utilization` default 0.92). https://github.com/vllm-project/vllm/blob/main/vllm/engine/arg_utils.py
- vLLM documentation, "Tool Calling" (parser names, `--enable-auto-tool-choice`, named and required `tool_choice` via structured outputs, Llama 3 without and Llama 4 with parallel calls). https://docs.vllm.ai/en/stable/features/tool_calling/
- Kurtic et al., "'Give Me BF16 or Give Me Death'? Accuracy-Performance Trade-Offs in LLM Quantization," 2024 (over 500,000 evaluations across Llama 3.1 8B, 70B and 405B; FP8 lossless; INT8 1 to 3 percent; W4A16 for synchronous, W8A8 for asynchronous serving). <https://arxiv.org/abs/2411.02355>
- Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," 2022. <https://arxiv.org/abs/2210.17323>
- Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," 2023. <https://arxiv.org/abs/2306.00978>
- Xiao et al., "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," 2022. <https://arxiv.org/abs/2211.10438>
- Dong et al., "XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models," 2024 (1,134 of 128K Llama 3.1 tokens context-dependent under a JSON grammar; under 40 microseconds per token; over 100× speedup). <https://arxiv.org/abs/2411.15100>
- Dean and Barroso, "The Tail at Scale," Communications of the ACM, 2013. <https://dl.acm.org/doi/10.1145/2408776.2408794>

- Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," OSDI 2024 (goodput under TT FT and TPOT SLOs). <https://arxiv.org/abs/2401.09670>
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP 2023. <https://arxiv.org/abs/2309.06180>